Control Flow in Python

Conditional Statements & Loops

Introduction to Control Flow

Control flow determines the order in which statements are executed. Python provides:

- Conditional Statements — execute code based on conditions (if, elif, else)
- Loops — repeat code multiple times (for, while)
- Control Statements — modify loop behavior (break, continue, pass)

Conditional Statements

- if — single condition
- if-else — two-way decision
- if-elif-else — multiple conditions
- Nested if — conditions inside conditions

Loops

- for — iterate over sequences
- while — condition-based repetition
- break — exit loop early
- continue — skip to next iteration

The if Statement

The simplest form of conditional. Executes code only if the condition is True.

Syntax

```
if condition:
    # code to execute
    # if condition is True

# code continues here
# regardless of condition
```

Example

```
age = 20

if age >= 18:
    print("You are an adult")

# Output:
# You are an adult
```

Key Points

- Indentation (4 spaces) is mandatory — defines the code block
- Colon (:) is required after the condition
- Comparison operators: ==, !=, <, >, <=, >=
- Logical operators: and, or, not

if-else Statement

Two-way decision making. One block executes if True, another if False.

Syntax

```
if condition:
    # code if True
else:
    # code if False
```

Example: Even or Odd

```
number = 7

if number % 2 == 0:
    print(f"{number} is even")
else:
    print(f"{number} is odd")

# Output:
# 7 is odd
```

Flow: condition → True → if block | False → else block

The else block is optional but must follow immediately after the if block.

if-else Statement

Use the **pass** statement if no statements exist for a particular clause:

Example:

```
number = 6

if number % 2 == 0:
    pass
else:
    print(f"{number} is odd")

# Output:
No output
```

if-elif-else Statement

Multiple conditions checked in order. First True condition wins.

Syntax

```
if condition1:
    # code 1
elif condition2:
    # code 2
elif condition3:
    # code 3
else:
    # default code
```

Example: Grade Calculator

```
score = 85

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
elif score >= 60:
    grade = "D"
else:
    grade = "F"

print(f"Grade: {grade}")  # Grade: B
```

Only the first matching condition executes. Order matters! Put most specific conditions first.

Nested if Statements

An if statement inside another if statement. Used for complex decisions.

```
age = 25
has_id = True

if age >= 18:
    print("Age check passed")
    if has_id:
        print("Entry granted")
    else:
        print("Please show ID")
else:
    print("Too young")

# Output:
# Age check passed
# Entry granted
```

⚠ Avoid deep nesting (more than 3 levels).
It makes code hard to read and maintain.

Use logical operators (and, or) to flatten nested conditions when possible.

The for Loop

Iterates over a sequence (list, string, range, etc.) and executes code for each item.

Syntax

```
for item in sequence:
    # code to execute
    # for each item
```

Examples

```
# Iterate over a list
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)

# Iterate over a string
for char in "Python":
    print(char)  # P, y, t, h, o, n
```

The for Loop

The range() Function — Generate Number Sequences

```
range(5)        # 0, 1, 2, 3, 4
range(2, 6)     # 2, 3, 4, 5
range(0, 10, 2) # 0, 2, 4, 6, 8  (step=2)

for i in range(5):
    print(i)  # 0 1 2 3 4
```

for Loop: enumerate() & zip()

Powerful built-in functions that enhance for loops.

enumerate() — Get Index + Value

```
fruits = ["apple", "banana", "cherry"]

for index, fruit in enumerate(fruits):
    print(f"{index}: {fruit}")

# Output:
# 0: apple
# 1: banana
# 2: cherry
```

for Loop: enumerate() & zip()

- **for i,x in enumerate(s):**
- *statements*

- enumerate(s) creates an iterator that simply returns a sequence of tuples (0, s[0]) , (1, s[1]) , (2, s[2]) , and so on

for Loop: enumerate() & zip()

Powerful built-in functions that enhance for loops.

zip() — Iterate Multiple Lists

```
names = ["Alice", "Bob", "Charlie"]
scores = [85, 92, 78]

for name, score in zip(names, scores):
    print(f"{name}: {score}")

# Output:
# Alice: 85
# Bob: 92
# Charlie: 78
```

for Loop: enumerate() & zip()

- # s and t are two sequences
- **for x,y in zip(s,t):**
- statements

- zip(s,t) combines sequences s and t into a sequence of tuples (s[0],t[0]) , (s[1],t[1]) , (s[2], t[2]) , and so forth, stopping with the shortest of the sequences s and t should they be of unequal length

Loop control statement - break and continue

Control statements that modify loop execution flow.

break — Exit Loop Immediately

```
# Find first negative number
numbers = [3, 7, -2, 9, 5]

for num in numbers:
    if num < 0:
        print(f"Found: {num}")
        break

# Output: Found: -2
# Loop stops immediately!
```

continue — Skip to Next Iteration

```
# Skip even numbers
for i in range(6):
    if i % 2 == 0:
        continue
    print(i)

# Output: 1 3 5
# Even numbers are skipped!
```

```
break    → Exit loop completely, go to code after loop
continue → Skip current iteration, go to next iteration
pass     → Do nothing (placeholder for future code)
```

else with for loops

The else block executes when the loop completes normally (without break).

Syntax

```
for item in sequence:
    if condition:
        break
else:
    # runs if loop did NOT break
    print("Loop completed!")
```

```
# Search with else
names = ["Alice", "Bob", "Charlie"]
for name in names:
    if name == "Dave":
        print("Found!")
        break
else:
    print("Dave not found")  # This runs!
```

Example: Prime Check

```
n = 17

for i in range(2, n):
    if n % i == 0:
        print(f"{n} is not prime")
        break
else:
    print(f"{n} is prime!")

# Output: 17 is prime!
# (else runs because no break)
```

else with for loops

- The else clause is confusing at first. Think of it as 'no break' clause.

- The else clause of a loop executes only if the loop runs to Completion.

- This either occurs immediately (if the loop wouldn’t execute at all) or after the last iteration.

- On the other hand, if the loop is terminated early using the break statement, the else clause is skipped.

The while Loop

Repeats code as long as a condition is True. Use when iteration count is unknown.

Syntax

```
while condition:
    # code to repeat
    # update condition

# code after loop
```

Example: Countdown

```
count = 5

while count > 0:
    print(count)
    count -= 1  # decrement

print("Blast off!")
```

```
# BAD: Infinite loop!
x = 1
while x > 0:
    print(x)
    # x never changes → runs forever!
```

⚠ Danger: Infinite Loops

Always ensure the loop condition will eventually become False. Use Ctrl+C to stop a runaway loop.

break and continue

Control statements that modify loop execution flow.

break — Exit Loop Immediately

```
# Find first negative number
numbers = [3, 7, -2, 9, 5]

for num in numbers:
    if num < 0:
        print(f"Found: {num}")
        break

# Output: Found: -2
```

continue — Skip to Next Iteration

```
# Skip even numbers
for i in range(6):
    if i % 2 == 0:
        continue
    print(i)

# Output: 1 3 5
# Even numbers are skipped!
```

Comparison

```
break    → Exit loop completely, go to code after loop
continue → Skip current iteration, go to next iteration
pass     → Do nothing (placeholder for future code)
```

else with while loops

The else block executes when the loop completes normally (without break).

Syntax

```
while condition:
    # execute these statements
else:
    # execute these statements
```

Example

```
num=12
while(i<10):
   if (num == I)
      print(“found”)
      Break
   i=i+1
else:
 print(“not found”)
```

Nested Loops

A loop inside another loop. Outer loop runs once, inner loop runs completely.

Example: Multiplication Table

```
for i in range(1, 4):
    for j in range(1, 4):
        print(f"{i}x{j}={i*j}", end=" ")
    print()  # new line

# Output:
# 1x1=1 1x2=2 1x3=3
# 2x1=2 2x2=4 2x3=6
# 3x1=3 3x2=6 3x3=9
```

Example: Star Pattern

```
rows = 5

for i in range(1, rows + 1):
    for j in range(i):
        print("*", end="")
    print()

# Output:
# *
# **
# ***
# ****
# *****
```

Nested loops can be slow for large data. Consider list comprehensions or built-in functions for optimization.

Summary & Quick Reference

Key takeaways for Python control flow:

Conditionals

- if — single condition check
- if-else — two-way decision
- if-elif-else — multiple conditions
- Nested if — conditions inside conditions
- Comparison: ==, !=, <, >, <=, >=
- Logical: and, or, not

Loops

- for — iterate over sequences
- while — condition-based repeat
- range(start, stop, step)
- enumerate() — index + value
- zip() — multiple iterables

- break — exit loop
- continue — skip iteration